

Évaluer les performances d'un système RAG

Rapport de mise en place et d'évaluation — Agent IA NBA « SportSee »

Auteur : Jacob Bessire
Date : 29/09/2026
Dépôt GitHub : github.com/jackobess/OC_Projet10
Stack : Mistral AI, FAISS, SQLite, LangChain, Pydantic AI, RAGAS, PydanticLogfire, Streamlit

Sommaire

1. Contexte et mission	3
2. Audit du prototype existant	3
3. Méthodologie d'évaluation RAGAS	4
4. Évaluation initiale du prototype	5
5. Mise en place des améliorations	6
6. Évaluation finale et comparatif avant / après	7
7. Limites et axes d'amélioration	9
8. Conclusion	9

1. Contexte et mission

SportSee dispose d'un prototype d'assistant IA fonctionnel reposant sur une architecture RAG (Retrieval-Augmented Generation). Ce prototype donne des résultats encourageants sur les archives textuelles (threads Reddit) mais ne permet pas de répondre précisément aux questions portant sur les statistiques chiffrées des joueurs et des équipes NBA.

L'objectif de ce projet est double : auditer et évaluer le système existant, puis le faire évoluer pour qu'il devienne fiable sur l'ensemble des types de questions posées par les utilisateurs. Le travail s'est déroulé en cinq étapes :

1. Audit et évaluation RAGAS du système existant ;
2. Amélioration du pipeline de préparation des données et d'indexation (chunking, index vectoriel) ;
3. Amélioration de l'agent, notamment par la création d'un outil LangChain de requêtage SQL pour les questions statistiques ;
4. Évaluation RAGAS du système amélioré et comparaison avec l'évaluation initiale ;
5. Identification des limites et des axes d'amélioration.

2. Audit du prototype existant

2.1 Les données sources

Threads Reddit (PDF)

- Les threads sont fournis sous forme d'images, ce qui impose un passage par OCR. Le prototype initial utilisait `EasyOCR`, peu performant : temps d'exécution élevé et qualité de texte extrait médiocre.
- Seuls 4 threads (4 fichiers Reddit) sont disponibles, ce qui constitue un volume de données très limité.

Fichier Excel de statistiques

- Une seule saison couverte (2024-2025).
- Uniquement des statistiques agrégées par joueur sur l'ensemble de la saison — impossible par exemple de répondre à une question sur les derniers matchs joués.
- Aucune distinction entre les statistiques à domicile et à l'extérieur.

2.2 Le pipeline de préparation des données

Le pipeline d'ingestion initial (`indexer.py`) présentait plusieurs faiblesses structurelles :

- **Manque de modularité** : un unique script principal (`indexer.py`), appuyé sur deux modules `data_loader.py` et `vector_store.py`) enchaînait toutes les étapes, de la récupération des sources jusqu'à l'indexation vectorielle.
- **OCR sous-performant** : EasyOCR restait lent et produisait un texte de qualité insuffisante.
- **Perte de structure** : toutes les sources, y compris le fichier Excel, étaient converties en texte brut avant découpage en chunks, avec un risque de perte d'information structurée.
- **Duplication de configuration** : coexistence d'un fichier `.env` et d'un `config.py` , entraînant une duplication des paramètres (modèles, répertoire d'entrée, index FAISS, etc.).

2.3 L'agent IA (MistralChat.py)

- **Absence de base de données statistiques** : toutes les requêtes passaient par une recherche de similarité vectorielle sur les chunks, y compris les questions chiffrées — d'où des réponses peu fiables sur les statistiques.
- **Prompt minimal et peu contraignant** : l'agent pouvait ignorer le contexte fourni et « halluciner » librement.
- **Prompt codé en dur** dans `MistralChat.py` , donc peu accessible et peu modifiable par un profil non technique.

3. Méthodologie d'évaluation RAGAS

3.1 Catégories de questions

Un jeu de questions d'évaluation (`ragas_dataset.py`) a été construit autour de cinq catégories, afin de couvrir les différents cas d'usage et points de faiblesse identifiés lors de l'audit :

Catégorie	Définition	Source du ground truth
simple	Question factuelle directe, réponse attendue dans un seul chunk Reddit	PDF Reddit 1 à 4
complexe	Nécessite de croiser plusieurs commentaires / threads	PDF Reddit 1 à 4
chiffrée	Agrégation calculable sur <code>regular_NBA.xlsx</code>	Statistiques Excel
hors_scope	Question dont la réponse n'existe pas dans les données disponibles	Réponse attendu pour le hors-scope
bruitée	Cible une section de document où du bruit a été repéré (publicité, fin de page...)	PDF Reddit 1 à 4

3.2 Les cinq métriques RAGAS

Faithfulness (fidélité) — la réponse est-elle fidèle aux documents récupérés ? On vérifie que chaque affirmation de la réponse est appuyée par le contexte (score = affirmations supportées / total).

Answer Relevancy (pertinence) — la réponse répond-elle bien à la question posée ? Des questions sont régénérées à partir de la réponse puis comparées à la question d'origine ; pénalise les réponses hors sujet, incomplètes ou évasives.

Context Precision (précision du contexte) — les passages pertinents sont-ils bien classés en tête des résultats de retrieval ? Mesure la qualité et le « bruit » de la recherche.

Context Recall (rappel du contexte) — le retrieval a-t-il ramené toute l'information nécessaire ? Vérifie que chaque élément de la réponse de référence se retrouve dans les contextes récupérés.

Answer Correctness (exactitude) — la réponse est-elle juste par rapport à la réponse de référence ? Combine 75 % d'exactitude factuelle (F1 sur les affirmations) et 25 % de similarité sémantique.

En synthèse : Context Precision et Context Recall évaluent le *retrieval*, Faithfulness et Answer Relevancy évaluent la *génération*, Answer Correctness évalue le résultat *bout en bout*.

Remarque méthodologique : le jeu de test comporte 9 questions au total, réparties sur 5 catégories (1 à 2 questions par catégorie). Les scores par catégorie doivent donc être lus comme des indicateurs qualitatifs plutôt que comme des moyennes statistiquement robustes : un seul cas particulier peut faire basculer un score de catégorie de 0 à 1.

4. Évaluation initiale du prototype

La première évaluation RAGAS a été menée sur le prototype tel qu'audit, avant toute modification du pipeline ou de l'agent.

4.1 Scores globaux

Métrique	Score
Faithfulness	0.359
Answer Relevancy	0.546
Context Precision	0.559
Context Recall	0.472
Answer Correctness	0.322

4.2 Scores par catégorie

Catégorie	Faithfulness	Relevancy	Precision	Recall	Correctness
bruitée	0.333	0.000	0.750	0.00	0.479
chiffree	0.489	0.411	0.271	0.25	0.252
complexe	0.143	0.850	1.000	0.50	0.446
hors_scope	0.000	0.783	0.600	1.00	0.198
simple	0.800	0.857	1.000	0.75	0.570

4.3 Lecture des résultats

Le score global de **faithfulness (0.359)** et d'**answer correctness (0.322)** confirment le diagnostic de l'audit : l'agent s'appuie insuffisamment sur le contexte récupéré et produit des réponses peu ancrées dans les documents. Deux points se dégagent en particulier :

- La catégorie **chiffree** obtient les scores les plus faibles sur le retrieval (context precision 0.271, recall 0.25) : sans base de données requêteable, les questions statistiques sont traitées par similarité vectorielle sur des chunks textuels, un mode de recherche mal adapté aux données tabulaires.
- La catégorie **bruitée** illustre l'effet direct de l'OCR de mauvaise qualité : answer relevancy à 0.000 et faithfulness à 0.333, le bruit du texte extrait dégradant la génération.

5. Mise en place des améliorations

5.1 Refonte du pipeline de préparation des données

Le pipeline monolithique initial a été remplacé par trois scripts modulaires et indépendants :

`step1_prepare_data.py` — **Extraction et nettoyage**

- Remplacement d'EasyOCR par **RapidOCR** plus rapide et plus précis sur les captures de threads Reddit.
- Extraction du texte des PDF puis nettoyage via un agent **Pydantic AI** chargé de supprimer le bruit (publicités, fins de page, etc.).
- Ne traite que les sources documentaires (.pdf, .doc, .txt) — le fichier Excel est traité séparément (voir step 3) afin de préserver sa structure tabulaire.
- Validation Pydantic des données extraites avant passage à l'étape suivante.

`step2_indexer.py` — **Indexation vectorielle**

- Chunking du texte nettoyé et indexation dans un index vectoriel **FAISS**.
- Sortie : `faiss_index.idx` et `chunks.pkl`.

`step3_excel_to_db.py` — **Conversion en base de données**

- Conversion du fichier Excel de statistiques en base **SQLite** requêtable (`nba.db`), plutôt que de le faire transiter par le pipeline texte.
- Validation Pydantic de chaque ligne avant insertion.

Ce découpage rend le pipeline plus lisible, plus efficace, et surtout plus sûr : chaque étape est testable indépendamment et produit un artefact clairement identifié (index vectoriel d'un côté, base de données statistiques de l'autre).

5.2 Amélioration de l'agent IA

- **Extraction des prompts** hors du code, dans `utils/prompts.py`, avec des règles de réponse plus strictes (cadrage sur le contexte, gestion explicite des cas hors sujet ou hors saison).
- **Intégration d'un outil de requête SQL** via LangChain (`utils/sql_tools.py`), permettant de router les questions statistiques vers la base `nba.db` plutôt que vers la recherche vectorielle.
- **Amélioration de l'interface** Streamlit pour faciliter les tests et le débogage.
- **Intégration de Pydantic Logfire** pour le monitoring et le troubleshooting (traces sur le retrieval, le routage SQL et la génération).

6. Évaluation finale et comparatif avant / après

6.1 Scores globaux — comparatif

Métrique	Avant	Après	Écart
Faithfulness	0.359	0.663	+0.304
Answer Relevancy	0.546	0.601	+0.055
Context Precision	0.559	0.633	+0.074
Context Recall	0.472	0.800	+0.328
Answer Correctness	0.322	0.926	+0.604

6.2 Scores par catégorie — comparatif

Catégorie	Faithf.	Relev.	Précis.	Recall	Correct.
bruitée — avant	0.333	0.000	0.750	0.00	0.479
bruitée — après	1.000	0.946	0.867	1.000	0.869
chiffrée — avant	0.489	0.411	0.271	0.25	0.252
chiffrée — après	0.950	0.881	1.000	1.000	0.963
complexe — avant	0.143	0.850	1.000	0.50	0.446
complexe — après	0.833	0.831	0.833	1.000	0.540
hors_scope — avant	0.000	0.783	0.600	1.00	0.198
hors_scope — après	0.167	0.000	0.000	0.333	1.000
simple — avant	0.800	0.857	1.000	0.75	0.570
simple — après	0.500	0.713	1.000	1.000	1.000

6.3 Analyse des résultats

Sur l'ensemble des cinq métriques globales, les gains sont nets, en particulier sur l'**answer correctness** (+0.604) et le **context recall** (+0.328). Deux évolutions expliquent l'essentiel du progrès :

- **La catégorie chiffrée** passe d'un profil de scores faibles et hétérogènes à un profil quasi optimal (précision et recall à 1.000, correctness à 0.963). C'est l'effet direct du remplacement du retrieval vectoriel par l'outil SQL sur la base `nba.db` : les questions statistiques sont désormais traitées par une requête exacte plutôt que par une recherche approximative de similarité.
- **La catégorie bruitée** progresse fortement sur toutes les métriques, confirmant que le passage à RapidOCR et le nettoyage par agent Pydantic AI ont bien réduit le bruit qui polluait auparavant la génération (answer relevancy 0.000 → 0.946).

Deux résultats à nuancer.

La catégorie **hors_scope** voit sa relevancy et sa précision chuter à 0.00 après amélioration, alors que sa correctness atteint 1.00.

- ⇒ la réponse standardisée de refus (imposée par les nouveaux prompts stricts, `utils/prompts.py`) obtient une correctness parfaite mais une similarité sémantique jugée faible face à la question posée, ce qui pénalise mécaniquement relevancy et precision malgré une réponse « correcte »

La catégorie **simple** voit sa faithfulness reculer de 0.80 à 0.50 alors que ses autres métriques progressent jusqu'à 1.00.

- ⇒ Pour ce cas précis la réponse est bien trouvé dans le contexte (2 prénoms de joueurs) mais le llm complete la réponse en ajoutant les noms de famille.

La catégorie **complexe** progresse nettement sur faithfulness (0.143 → 0.833) et recall (0.50 → 1.000), mais reste la plus faible en answer correctness (0.540) : croiser plusieurs commentaires ou threads pour construire une réponse reste l'exercice le plus difficile pour l'agent, ce qui rejoint le constat fait en conclusion sur la nécessité d'un pipeline runtime plus sophistiqué.

7. Limites et axes d'amélioration

- **Volume de données de référence limité** : davantage de sources (Reddit ou autres) et de données statistiques (match par match, plusieurs saisons) permettraient une évaluation plus robuste et un agent plus complet.
- **OCR sans structure de discussion** : l'extraction ne tient pas compte de la hiérarchie des commentaires dans un thread Reddit — les posts peuvent se retrouver décorrélés de leur contexte — ni de la popularité des posts (upvotes/downvotes), qui serait un signal utile de pertinence.
- **Pipeline runtime à sophistiquer** : certaines questions nécessitent un enchaînement RAG puis SQL. Par exemple, « parmi les équipes favorites des fans, lesquelles ont la meilleure défense ? » suppose d'abord d'identifier les équipes favorites des fans dans le contexte RAG avant d'interroger l'outil SQL — un cas non géré par le routage actuel, à un seul niveau.
- **Marge de progression sur les prompts**, en particulier pour mieux cadrer les cas limites (commentaires d'analyste, réponses hors scope).
- **Absence de mémoire de session** : chaque question est traitée indépendamment de la précédente, sans prise en compte de l'historique de la conversation.

8. Conclusion

L'audit initial a mis en évidence deux faiblesses structurelles majeures : un OCR peu performant dégradant la qualité des données textuelles, et l'absence de toute base de données statistiques obligeant à traiter les questions chiffrées par similarité vectorielle. La refonte du pipeline de préparation (RapidOCR, nettoyage par agent Pydantic AI, séparation des flux texte/statistiques) et l'ajout d'un outil SQL dédié à l'agent ont permis des gains substantiels sur l'ensemble des métriques RAGAS, en particulier sur l'answer correctness (+0.604) et le context recall (+0.328).

Les scores par catégorie confirment que les gains les plus nets concernent précisément les deux points identifiés à l'audit — catégories *bruitée* et *chiffrée*. Les questions *complexes*, nécessitant de croiser plusieurs sources, restent le point faible du système et constituent, avec l'enrichissement du jeu de données et la gestion de l'historique de conversation, les priorités identifiées pour la suite du projet.